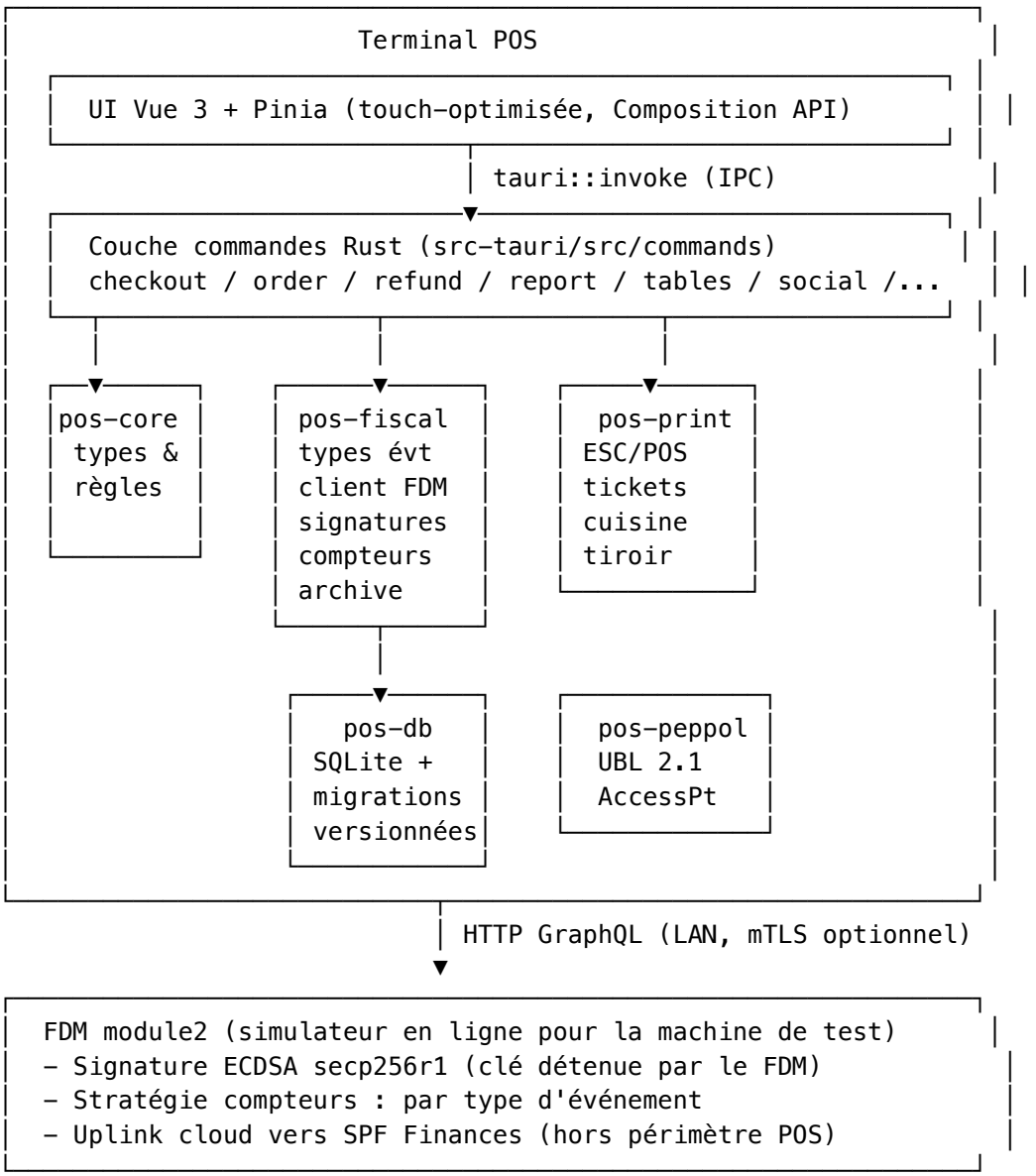
Document de processus technique - pili-pos SCE 2.0

Distributeur : Bruno Moyse / nuagemagique.dev (Bruxelles, BE) **Nom du produit :** pili-pos **Version :** 1.0.0 (build de soumission cert) **Clients cibles :** établissements HoReCa en Belgique (restaurants avec service en salle) **Date de soumission :** 2026-05-14 **Specs implémentées :** POS v2.2 + FDM v1.4 + Addendum 1 + Basic Set v1.7 + Reports v1.3 + Advanced Set v1.5

1. Vue d'ensemble du système

`pili-pos` est une **application desktop Tauri v2** s'exécutant sur Linux x86_64 (Debian 13 minimal + compositeur Cage Wayland en mode kiosk). Elle cible les terminaux POS Penetek A3216 M82 / Advantech USC-360J, se connecte à un FDM certifié SPF Finances par LAN, pilote une imprimante thermique ESC/POS Epson TM-m30III, et persiste chaque événement fiscal dans une base SQLite locale append-only.



2. Comportement par événement

Événement	Déclencheur	Chemin de code POS	Notes
N (Vente normale)	L'opérateur tape "Payer" avec `order_type=dine_in`	`commands/checkout::checkout` → `fdm_client::sign_sale` → `archive` + close dans une seule tx SQLite	Blocage-sur-déconnexion-FDM imposé
P (Pro forma)	L'opérateur envoie la commande en cuisine	`commands/tables::sign_table_order`	Table ouverte ; pas d'incrément du compteur N
T (Formation)	L'opérateur est en mode formation	Comme N mais `is_training=true` dans l'input ; watermark UI	
C (Copie)	Réimpression d'un ticket antérieur	`commands/copy::copy_ticket`	Porte un `fdmRef` vers l'original
I (Facture)	Mode de paiement "Facture" choisi	`commands/checkout` → `sign_invoice`	Si `customer_vat_no` renseigné + Peppol activé → `commands/invoice::issue_b2b_invoic`
F (Financier)	Mouvement caisse, ouverture seule du tiroir	`commands/financial::money_in_out` / `open_drawer_fiscal`	Fond de caisse ouverture tiroir
S (Social)	Pointage entrée / sortie de l'opérateur	`commands/social::work_in` / `work_out`	INSZ + n° employeur
R (Rapport)	Rapports Z et X	`commands/report::generate_x_report` / `generate_z_report`	Z-test-Turnover-Z + User-Z atomique

Chaque branche :

1. Charge les lignes pertinentes depuis SQLite (les requêtes sont dans `pos-db/src/queries/`).
2. Construit la struct d'input SCE-2.0 (`pos-fiscal::inputs::*`).
3. Calcule la clé d'idempotence (voir §3.4) et le corps JSON canonique (voir §3.3).
4. Appelle la méthode async `FdmClient::sign_*` correspondante.
5. Sur succès : ajoute à `fiscal_events` + ferme/marque la commande dans une seule transaction SQLite.
6. Retourne les données de ticket à l'UI ; l'UI déclenche l'impression via `pos-print`.

3. Signature / vérification

3.1 Transport

- **Protocole :** GraphQL sur HTTP/HTTPS.
- **Endpoint :** configuré via `pos.json` (`fdm_url`).
- **Sondage de statut :** `FdmClient::status` est appelé au démarrage de l'app et à chaque entrée de checkout.

3.2 Signature de réponse

- Algorithme : **ECDSA secp256r1**, calculé par le FDM avec sa propre clé. Le POS ne détient pas la clé publique du FDM et ne vérifie pas la signature ECDSA ; il la conserve telle quelle, avec la réponse complète, dans `fiscal_events.fdm_response_json`.
- Contrôle de cohérence avant archivage : `pos_fiscal::response_check::check_sign_result`, appelé par `FdmClient` pour chaque réponse de signature (y compris les deux moitiés du Z M184). Il exige l'écho de `posId`, `posFiscalTicketNo`, `posDateTime`, `deviceId` et `terminalId`, l'`eventOperation` et l'`eventLabel` de la mutation (T en formation), une `digitalSignature` en base64 valide et, pour N, une `shortSignature` égale à Hex(SHA1(Base64Decode(digitalSignature)))

(pos_fiscal::signature::compute_short_signature) ainsi qu'une verificationUrl. Toute divergence produit l'erreur FDM_RESPONSE_MISMATCH : rien n'est archivé ni imprimé.
- La short signature apparaît sur le ticket imprimé (40 caractères hexadécimaux).

3.3 JSON canonique

- pos-fiscal::canonical::to_canonical() sérialise une valeur en JSON canonique style RFC 8785 : clés triées par codepoint Unicode à chaque niveau, pas d'espace insignifiant, nombres normalisés (pas de zéros traînants, $-0 \rightarrow 0$).
- Utilisé pour **chaque** corps de requête envoyé au FDM, de sorte qu'un flux de signature reproductible par hash est possible.
- Test de propriété dans crates/pos-fiscal/src/canonical.rs::tests (vérification de déterminisme sur 50 itérations).

3.4 Clé d'idempotence

- pos-fiscal::inputs::IdempotencyKey { pos_id, pos_date_time, terminal_id, event_label, pos_fiscal_ticket_no }.
- Chaque type d'input implémente .idempotency_key().
- Permet le retry sûr sur les échecs FDM transitoires : une requête dupliquée avec la même clé est reconnue par le FDM comme une re-soumission et renvoie la signature originale.
- Les tests unitaires dans crates/pos-fiscal/src/inputs.rs confirment que deux requêtes sémantiquement différentes au même ticket number déclenchent une erreur de réutilisation de compteur avant l'envoi.

4. Stratégie de compteurs

Voir STRATEGIE_COMPTEURS_FR.md. **Compteurs par type d'événement** stockés dans la table fiscal_counters ; alloués sous BEGIN IMMEDIATE à l'intérieur de la même transaction SQLite qui ajoute à fiscal_events, de sorte que :

- Les compteurs sont strictement monotones par type d'événement.
- Un appel FDM crashé rollback toute la transaction ; le compteur est libéré - pas de trou.
- Au retry, le même (event_type, pos_fiscal_ticket_no) est présenté au FDM, qui honore l'idempotence.

5. Comportement hors-ligne / déconnexion

- **Pas de FDM, pas d'événement.** Chaque commande fiscale (vente, remboursement, commande, addition, copie, pointage, mouvement de caisse, ouverture de tiroir, rapport) fait signer son événement par le FDM *avant* d'écrire quoi que ce soit : si le FDM ne répond pas, la commande renvoie une erreur (success: false, « FDM not connected - cannot complete transaction » pour une vente), rien n'est archivé, aucun compteur n'avance, la commande reste ouverte et le tiroir reste fermé. L'opérateur réessaie une fois le FDM revenu.
- **Aucun mode hors-ligne**, dans aucune build : le code ne contient aucun chemin qui archive ou imprime une vente non signée.
- L'en-tête affiche en permanence l'état du FDM (FdmStatus.vue, interrogé toutes les 30 s), avec un message lisible pour chaque code d'erreur du FDM.
- Seule exception : les entrées de commande (P) non encore envoyées restent en attente dans order_entries et partent à l'enregistrement suivant (commande, addition ou vente) ; ce ne sont pas des événements tant que le FDM ne les a pas signées.

6. Pipeline d'impression

- Crate : crates/pos-print.

- Transport : `pos-print::transport` supporte TCP, USB et série. La configuration de référence utilise TCP vers la TM-m30III à une IP LAN statique.
- Layout : `pos-print::receipt` formate le ticket fiscal SCE-2.0 selon la spec POS v2.2 §III.3 - en-tête (nom restaurant, adresse, TVA), corps (lignes avec label TVA), table de répartition TVA, bloc de données de contrôle issu de la réponse FDM, QR Code (Model 2, ECC M, URL 38 caractères ; version 3, 29×29 modules - voir `TICKET_NUMERIQUE_FR.md` §11).
- Tickets cuisine : layout séparé (`crates/pos-print/src/kitchen_ticket.rs::format_kitchen_ticket`).
- Ouverture tiroir : séquence ESC/POS `0x1B 0x70 0x00 0x19 0xFA` émise dès qu'un paiement `cash` clôture une vente.

6bis. Pipeline d'envoi du ticket numérique (e-mail)

- Crate : `crates/pos-mail`, plus les tables `ticket_deliveries` et `original_prints` de `crates/pos-db` (migrations `005`/`006`).
- Rendu (`pos-mail::render`) : corps texte = `pos_print::render` appliqué au **même** `format_receipt` que le papier ; corps HTML + QR PNG en pièce jointe uniquement quand le FDM a renvoyé une URL de vérification (événements N).
- Transport (`pos-mail::tem`) : client HTTP vers l'API Scaleway Transactional Email (`X-Auth-Token`, région `fr-par` par défaut).
- File d'attente (`pos-mail::outbox`) : tentative immédiate bornée à 8 s à l'encaissement, puis réessais en tâche de fond avec recul exponentiel (`commands::delivery::run_outbox_once`, toutes les 30 s). Détail complet : `TICKET_NUMERIQUE_FR.md`.

6ter. Commandes en ligne

- Crates : `crates/pos-hubrise` (réception, file d'envoi, menu reflet), `crates/pos-catalog` (validation et application du catalogue), commandes Tauri `src-tauri/src/commands/webshop.rs`.
- Interrupteur par installation : `hubrise_enabled` dans `pos.json`, puis un Gérant dans Réglages ; journal en ajout seul `integration_switch_log` (migration 25). Coupé : aucune boucle ne contacte HubRise et aucune commande n'est acceptée ; une commande déjà signée reste soldable.
- Chaque commande en ligne se termine par un N signé sur ce terminal, au nom de l'employé qui l'accepte (P puis N si elle est payée à la remise). Détail : `COMMANDES_EN_LIGNE_FR.md`.

7. Persistance

- Moteur : **SQLite** (DB en un seul fichier dans `~/.local/share/pili-pos/pos.db` en dev, `/var/lib/pili-pos/pos.db` en prod).
- Mode : WAL (`PRAGMA journal_mode=WAL`).
- Migrations : versionnées via `pos-db::migrate::run_migrations`. La table `_migrations_applied` suit `(version, name, applied_at)`. Le chemin de rétro-compatibilité pour la migration one-shot d'origine est dans `migrate::backfill_legacy_db`.
- Log fiscal append-only : table `fiscal_events`. Le schéma impose la contrainte CHECK sur `event_type`, et le code applicatif n'émet jamais `DELETE` ni `UPDATE` à son encontre.
- Sauvegarde : snapshot nocturne via l'API SQLite Online Backup (`commands::report::backup_database`) avant chaque rapport Z.

8. Configurations de build

Build	Features Cargo	Produit
Dev	(par défaut) `hubrise` + `peppol`	App complète, facturation B2B comprise

Build	Features Cargo	Produit
Cert	`--no-default-features --features cert` (hubrise + interface embarquée)	Binaire certifié : commandes en ligne présentes, activées par installation (`COMMANDES_EN_LIGNE_FR.md`) ; sans facturation Peppol (hors v1.0.0)

Vérification sur le binaire cert (`scripts/build_and_hash.sh`) :

```
$ strings target/release/pili-pos | grep -c integration_switch_log  # commandes en ligne présentes
$ nm target/release/pili-pos | grep -c pos_peppol                   # facturation B2B absente
0
```

9. Carte des sources

Préoccupation	Crate / fichier
Types d'événements fiscaux & structs d'input	`crates/pos-fiscal/src/inputs.rs`
Client GraphQL FDM	`crates/pos-fiscal/src/fdm_client.rs`
JSON canonique	`crates/pos-fiscal/src/canonical.rs`
Short signature (SHA-1)	`crates/pos-fiscal/src/signature.rs`
Contrôle des réponses FDM	`crates/pos-fiscal/src/response_check.rs`
Compteurs fiscaux	`crates/pos-fiscal/src/counter.rs`
Archivage tickets	`crates/pos-fiscal/src/archive.rs`
Schéma base de données	`crates/pos-db/src/schema.rs`
Runner de migrations	`crates/pos-db/src/migrate.rs`
CRUD commandes	`crates/pos-db/src/queries/orders.rs`
Helpers requêtes rapport Z	`crates/pos-db/src/queries/reports.rs`
Flux checkout	`src-tauri/src/commands/checkout.rs`
Tables / split / merge	`src-tauri/src/commands/tables.rs`
Remboursement + note de crédit	`src-tauri/src/commands/refund.rs`
Rapports (X/Z)	`src-tauri/src/commands/report.rs`
Social (événements S)	`src-tauri/src/commands/social.rs`
Financier (événements F)	`src-tauri/src/commands/financial.rs`
Copie (événements C)	`src-tauri/src/commands/copy.rs`
Sérialiseur UBL Peppol	`crates/pos-peppol/src/ubl.rs`
AccessPoint Peppol	`crates/pos-peppol/src/access_point.rs`
IPC Peppol	`src-tauri/src/commands/invoice.rs`
Résolution du support + IPC ticket numérique	`src-tauri/src/commands/delivery.rs`
Rendu e-mail (texte/HTML/QR)	`crates/pos-mail/src/render.rs`
Client Scaleway TEM	`crates/pos-mail/src/tem.rs`
File d'attente + réessais	`crates/pos-mail/src/outbox.rs`
Validation/masquage d'adresse	`crates/pos-mail/src/address.rs`
Requêtes `ticket_deliveries` / `original_prints`	`crates/pos-db/src/queries/{deliveries,prints}.rs`
Impression ESC/POS	`crates/pos-print/src/escpos.rs`
Layout ticket	`crates/pos-print/src/receipt.rs`
Transport imprimante	`crates/pos-print/src/transport.rs`